Intermediate | Graph Neural Networks | Highlights | IBM Technology

Graph Neural Networks: How GNNs Learn From Connected Data

Understand how message passing works and how GCN, GraphSAGE, GAT, GIN, and graph transformers each aggregate neighbor information differently.

For people comfortable with basic neural networks (weights, activations, matrix multiplication) who want to understand how models learn from graph-structured data · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

Watch the source video | Read the full lesson | Read the condensed version

What matters

- ✓ A graph pairs entities (nodes) with relationships (edges); an adjacency matrix records which nodes connect, and for directed graphs, in which direction.
- ✓ GNNs learn through message passing: every node repeatedly gathers, aggregates, and folds in information from its neighbors to update its own embedding.
- ✓ Stacking message-passing layers grows a node's receptive field — layer 1 reaches immediate neighbors, layer 2 reaches neighbors of neighbors, and so on.
- ✓ GNN architectures mainly differ in their aggregation rule: GCN smooths neighbor features, GraphSAGE samples neighbors to stay scalable, GAT learns per-neighbor attention weights, and GIN sums for maximum expressive power.
- ✓ Sum aggregation (used by GIN) preserves more structural detail than mean or max, which is why GIN matches the discriminative power of the Weisfeiler-Lehman (WL) graph isomorphism test while GCN-style mean/max pooling does not.
- ✓ Graph transformers let every node attend to every other node directly (not just immediate neighbors), and add a structural bias term to the attention score so the graph's topology isn't lost.
- ✓ Choosing an architecture is a tradeoff between expressive power (GIN), scalability to huge graphs (GraphSAGE), interpretable importance weighting (GAT), and long-range reasoning (graph transformers).

What it covers

01 **Graphs as data: nodes, edges, and the adjacency matrix** `0:00`
A graph represents entities as nodes and their relationships as edges; an adjacency matrix is a simple way to record exactly which nodes connect to which.

02 **Embeddings, and heterogeneous vs. homogeneous graphs** `2:10`
GNNs convert nodes (and edges, and whole graphs) into dense embedding vectors, and graphs are classified by whether they contain one type of node/edge or several.

03 **Message passing: how nodes share information** `2:51`
Nodes don't predict in isolation — in each layer, every node sends, aggregates, and folds in messages from its neighbors, and stacking layers widens how far that information travels.

04 **Graph Convolutional Networks (GCN): smoothing over neighbors** `5:03`
GCNs generalize the weight-sharing idea behind CNNs to graphs: every node gets a smoothed, aggregated version of its neighbors' embeddings, passed through a shared weight matrix and a non-linearity.

05 **GraphSAGE: sampling neighbors to scale to huge graphs** `6:25`

GraphSAGE ("sample and aggregate") learns to aggregate over a fixed-size sample of each node's neighbors instead of the whole neighborhood, keeping cost constant on graphs with millions of nodes.

06 **Graph Attention Networks (GAT): weighing neighbors unequally** `7:07`

GATs learn an attention coefficient for each neighbor so the model can focus more on the connections that matter, instead of treating every neighbor the same.

07 **Graph Isomorphism Networks (GIN) and why sum beats mean/max** `8:31`

GIN sums neighbor embeddings and pushes the result through an MLP, and that sum is what lets it tell apart graph structures that mean- or max-based GNNs incorrectly treat as identical.

08 **Graph Transformers: global attention over the whole graph** `11:31`

Graph transformers let any node attend to any other node directly, using standard query/key/value attention plus a structural bias term so the graph's topology still shapes the result.

09 **Choosing an architecture** `16:03`

Every GNN shares the same message-passing skeleton; the choice of architecture comes down to what you're optimizing for — smoothness, scale, interpretability, expressiveness, or long-range reasoning.

Glossary

Graph — A structure made of nodes (entities) and edges (relationships between pairs of nodes), formally written as G = (V, E).

Node / vertex — A single entity in a graph, such as a person, atom, or web page.

Edge — A connection between two nodes, representing a relationship; can be directed (one-way) or undirected (two-way).

Adjacency matrix — An N×N matrix where entry (i, j) is 1 if node i connects to node j, and 0 otherwise; a complete numeric encoding of a graph's structure.

Embedding — A dense, low-dimensional vector representation of a node, edge, or graph that captures both its features and its structural context.

Homogeneous graph — A graph with exactly one type of node and one type of edge.

Heterogeneous graph — A graph with more than one type of node and/or edge, such as a graph with both "student" and "teacher" nodes.

Message passing — The core GNN mechanism where each node repeatedly gathers information from its neighbors (create), combines it (aggregate), and folds it into its own representation (update).

Aggregation — The operation that combines multiple incoming neighbor messages into a single vector, e.g. sum, mean, max, or attention-weighted combination.

Graph Convolutional Network (GCN) — A GNN that aggregates neighbor embeddings (typically via a normalized average) and passes the result through a shared weight matrix and non-linearity, analogous to a CNN's shared filters.

GraphSAGE — "Sample and aggregate" — a GNN that aggregates over a fixed-size random sample of each node's neighbors rather than the full neighborhood, so it scales to very large graphs.

Graph Attention Network (GAT) — A GNN that learns a per-neighbor attention coefficient, so some neighbors contribute more to a node's updated embedding than others.

Attention coefficient — A learned, softmax-normalized weight (denoted alpha) indicating how much one node's message should count when updating another node's embedding.

Graph Isomorphism Network (GIN) — A GNN that sums neighbor embeddings and passes the result through a multilayer perceptron (MLP); designed to match the discriminative power of the Weisfeiler-Lehman test.

Multilayer perceptron (MLP) — A small neural network made of one or more fully connected layers with non-linear activations between them.

Isomorphic graphs — Two graphs that are structurally identical once you allow relabeling their nodes — same connectivity pattern, just different names for the nodes.

Weisfeiler-Lehman (WL) test — A classical, purely combinatorial algorithm for checking whether two graphs are structurally identical (isomorphic), used as a benchmark for how expressive a GNN's aggregation can be.

Query, key, value (Q, K, V) — Three learned projections of a node's embedding used in attention: the query is what a node looks for, the key is what it offers for comparison, and the value is what gets passed along once attention weights are computed.

Residual connection — Adding a layer's input back to its output before further processing, which helps stabilize training in deep networks.

Graph transformer — A GNN that applies transformer-style global attention, letting any node attend to any other node, with a structural bias term added to the attention score to preserve graph topology.

Multi-head attention — Running several attention computations (heads) in parallel, each with its own Q/K/V projections, then concatenating and combining their outputs.